



K.L.E. Society's
K. L. E. INSTITUTE OF TECHNOLOGY, HUBBALLI-27
(Approved by AICTE, Affiliated to VTU, Belagavi and ISO 9001:2015 Certified)



Dept. of Electronics and Communication Engg.

Laboratory Manual

for

VI SEMESTER

Embedded System Design Laboratory

(BEC601)

Prepared by

Prof. Pramod V Rampur

Approved by

Dr. Manu T. M.

College Vision

One amongst top five percent of engineering colleges in Karnataka state by providing conducive teaching-learning environment, enabling transformation of young minds into competent engineers responsive to societal needs and values.

Department Vision

Prepare professionals embodied with competence and values to meet the changing needs in the field of Electronics and Communication Engineering.

Department Mission

- Providing state-of-the-art hardware and software infrastructure and qualified faculty to constantly update the students to meet the requirements of industry and research institutions.
- Inculcating theoretical, practical, design and development skills necessary for lifelong learning.
- Bridging the gap between academia and industry by industry-institute interactions.
- Imbibing human values and ethics in students to make them socially responsible.

Program Outcomes (POs)

Engineering Graduates will be able to:

1. **Engineering knowledge:** Apply the knowledge of mathematics, science, engineering fundamentals, and an engineering specialization to the solution of complex engineering problems.
2. **Problem analysis:** Identify, formulate, review research literature, and analyze complex engineering problems reaching substantiated conclusions using first principles of mathematics, natural sciences, and engineering sciences.
3. **Design/development of solutions:** Design solutions for complex engineering problems and design system components or processes that meet the specified needs with appropriate consideration for the public health and safety, and the cultural, societal, and environmental considerations.
4. **Conduct investigations of complex problems:** Use research-based knowledge and research methods including design of experiments, analysis and interpretation of data, and synthesis of the information to provide valid conclusions.
5. **Modern tool usage:** Create, select, and apply appropriate techniques, resources, and modern engineering and IT tools including prediction and modeling to complex engineering activities with an understanding of the limitations.
6. **The engineer and society:** Apply reasoning informed by the contextual knowledge to assess societal, health, safety, legal and cultural issues and the consequent responsibilities relevant to the professional engineering practice.
7. **Environment and sustainability:** Understand the impact of the professional engineering solutions in societal and environmental contexts, and demonstrate the knowledge of, and need for sustainable development.
8. **Ethics:** Apply ethical principles and commit to professional ethics and responsibilities and norms of the engineering practice.
9. **Individual and teamwork:** Function effectively as an individual, and as a member or leader in diverse teams, and in multidisciplinary settings.

10. **Communication:** Communicate effectively on complex engineering activities with the engineering community and with society at large, such as, being able to comprehend and write effective reports and design documentation, make effective presentations, and give and receive clear instructions.
11. **Project management and finance:** Demonstrate knowledge and understanding of the engineering and management principles and apply these to one's own work, as a member and leader in a team, to manage projects and in multidisciplinary environments.
12. **Life-long learning:** Recognize the need for, and have the preparation and ability to engage in independent and life-long learning in the broadest context of technological change.

Program Specific Outcomes (PSOs)

PSO.1: Analyze, design, build and test analog, digital communication and embedded systems for given specifications.

Pre-requisites:

- i. Fundamentals knowledge of Microcontroller or Microprocessor.
- ii. Knowledge about instruction sets of Microcontrollers.

Course Outcomes-CO

1. On the completion of this laboratory course, the students will be able to:
2. Describe the architectural features and instructions of 32-bit microcontroller ARM Cortex M3.
3. Apply the knowledge gained for Programming ARM Cortex M3 for different applications.
4. Understand the basic hardware components and their selection method based on the characteristics and attributes of an embedded system.
5. Understand the hardware software co-design and firmware design approaches.
6. Explain the need of real time operating system for embedded system applications.

PO Mapping

Course Outcomes-CO	1	2	3	4	5	6	7	8	9	10	11	12	PSO1
	Engineering knowledge	Problem analysis	Design/development of solutions	Conduct investigations of	Modern tool usage	The engineer and society	Environment and sustainability	Ethics	Individual and team work	Communication	Project management	Life-long learning	Analyze, design, build and test analog, digital communication and embedded systems for given specifications.
Describe the architectural features and instructions of 32-bit microcontroller ARM Cortex M3.	2	2	3		2								2
Apply the knowledge gained for Programming ARM Cortex M3 for different applications.	2	2	2		3								3
Understand the basic hardware components and their selection method based on the characteristics and attributes of an embedded system.	2	2	2		2								2
Understand the hardware software co-design and firmware design approaches.	2	2	3		3								3
Explain the need of real time operating system for embedded system applications.	2	2	3		3								3
Overall	2	2	2.5		2.5								2.5

Evaluation:

Students Assessment (IA)

Internal Assessments.	Lab IA AVG A	CIE B	Mini Project C	Final IA Marks (A+B+C)
	20	10	20	50/2 = 25

Course Content

Sl.NO	Experiments
1	Write a program to find the sum of the first 10 integer numbers.
2	Write an Assembly Language Program (ALP) to i) Multiply two 16-bit numbers. ii) Add two 32-bit numbers.
3	Write a program to find the factorial of a number.
4	Write a program to add an array of 16 bit numbers and store the 32 bit result in internal RAM.
5	Write a program to find the square of a number (1 to 10) using a look-up table.
6	Write a program to find the largest or smallest number in an array of 32 numbers.
7	Write a program to arrange a series of 32 bit numbers in ascending/descending order.
8	Write a program to count the number of ones and zeros in two consecutive memory locations.
9	Interface a Stepper motor and rotate it in clockwise and anti-clockwise direction.
10	Interface a DAC and generate Triangular and Square waveforms.
11	Display the Hex digits 0 to F on a 7-segment LED interface, with a suitable delay in between.
12	Interface a simple Switch and display its status through Relay, Buzzer and LED

V

CONTENT

**Expt.
No.**

Title

Page No.

1	Write a program to find the sum of the first 10 integer numbers.	1
2	Write an Assembly Language Program (ALP) to i) Multiply two 16-bit numbers. ii) Add two 32-bit numbers.	2
3	Write a program to find the factorial of a number.	4
4	Write a program to add an array of 16 bit numbers and store the 32 bit result in internal RAM.	5
5	Write a program to find the square of a number (1 to 10) using a look-up table.	6
6	Write a program to find the largest or smallest number in an array of 32 numbers.	7
7	Write a program to arrange a series of 32 bit numbers in ascending/descending order.	9
8	Write a program to count the number of ones and zeros in two consecutive memory locations.	13
9	Interface a Stepper motor and rotate it in clockwise and anti-clockwise direction.	16
10	Interface a DAC and generate Triangular and Square waveforms.	19
11	Display the Hex digits 0 to F on a 7-segment LED interface, with a suitable delay in between.	22
12	Interface a simple Switch and display its status through Relay, Buzzer and LED	23

PART_A

ASSEMBLY LEVEL PROGRAMS

Conduct the following experiments on an ARM CORTEX M3 evaluation board to learn Assembly

Language Program and using evaluation version of Embedded 'C' & Keil uVision-4 tool/compiler.

1. Write a program to find the sum of the first 10 integer

numbers

```
AREA ADDITION, CODE, READONLY
ENTRY                               ; Mark first instruction to execute
EXPORT __main
__main
    MOV r0, #10                     ; Counter (Integer number 10)
    MOV r1, #0                       ; Partial Sum
    MOV r2, #1                       ; First Number
    NEXT ADD r1, r1, r2              ; Add sum with the next number
    ADD r2, #1                       ; Increment the next integer
    SUBS r0, #1                      ; Decrement counter
    BNE NEXT                        ; Branch to the loop if not equal to Zero
STOP
    NOP
    END                             ; Mark end of
```

Result:

2. Write an Assembly Language Program (ALP) to i) Multiply two 16-bit numbers.

ii) Add two 32-bit numbers.

i) Multiply two 16-bit numbers.

```

/* PROGRAM TO MULTIPLY TWO 16BIT NUMBERS          */
/*      VALUE1:      0x1234          (IN R1) */
/*      VALUE2:      0x1234          (IN R2) */
/*      RESULT:      0x14B5A90       (IN R3) */

        AREA MULTIPLY, CODE, READONLY
        ENTRY                               ; Mark first instruction to execute
        EXPORT __main
__main
        LDR r1,=0x1234                      ; Store first number in r0
        LDR r2,=0x1234                      ; Store second number in r1
        MUL r3, r1, r2                      ; Multiplication
        L1 B L1                             ; Infinite loop to stop the execution
        NOP
        END                                ; Mark end of file

```

Result:

2. Write an Assembly Language Program (ALP) to

ii) Add two 32-bit numbers.

```

/* PROGRAM TO ADD TWO 32BIT NUMBERS              */
/*      VALUE1:      0x12341234          (IN R1) */
/*      VALUE2:      0x12341234          (IN R2) */
/*      RESULT:      0x24682468          (IN R3) */

        AREA ADDITION, CODE, READONLY
        ENTRY                               ; Mark first instruction to execute
        EXPORT __main
__main
        LDR r1,=0x12341234                  ; Store first number in r0
        LDR r2,=0x12341234                  ; Store second number in r1
        ADD r3, r1, r2                      ; Addition
        L1 B L1                             ; Infinite loop to stop the execution
        NOP
        END                                ; Mark end of file

```

Result:

3. Write a program to find the factorial of a number.

```

        AREA FACT, CODE, READONLY
ENTRY; Mark first instruction to execute
EXPORT _main
_main
        MOV r0, #5                ; Factorial Number
        MOV r1, #1                ; Result Register
        CMP r0, #0                ; Compare factorial num with zero
        BEQ STOP                 ; If zero stop
        MOV r1, r0                ; copy the number
NEXT    SUBS r0, #1               ; Decrement the number
        CMP r0, #0               ; Compare it with zero
        BEQ STOP                 ; If zero stop
        MUL r2, r1, r0            ; Multiply with the next number
        MOV r1, r2               ; Save the result
        B NEXT                   ; Branch to the loop
STOP
        NOP
        END                      ; Mark end of file
    
```

Result:

4. Write a program to add an array of 16-bit numbers and store the 32-bit result in internal RAM

```

                AREA ADDITION, CODE, READONLY
ENTRY           ; Mark first instruction to execute
EXPORT __main

__main
    LDR R0, =ARRAY
    MOV R1, #0
    MOV R4, #0
NEXT
    CMP R1, #6
    BEQ STORE
    LDRH R3, [R0], #2
    ADC R4, R4, R3
    ADD R1, R1, #1
    B NEXT
STORE
    LDR R5, =SUM
    STR R4, [R5]
STOP
    B STOP
ARRAY
    DCW 0x0001, 0x0002, 0x0003, 0x0004, 0x0005, 0x0006
AREA RESULT, DATA, READWRITE
SUM
    DCD 0
END
    
```

Result:

5. Write a program to find the square of a number (1 to 10) using look-up table.

```

AREA SQUARE, CODE, READONLY
ENTRY                ; Mark first instruction to execute
EXPORT __main
__main                ; Mark first instruction to execute
    LDR R0, = TABLE1    ; Load start address of Lookup table
    LDR R1, = 6           ; Load no whose square is to be find
    MOV R1, R1, LSL#0x2   ; Generate address corresponding to square of given no
    ADD R0, R0, R1; Load address of element in Lookup table
    LDR R3, [R0]          ; Get square of given no in R3
    NOP                  ; Lookup table contains Squares of no's from 0 to 10

TABLE1                DCD 0X00000000    ; SQUARE OF 0=0
                    DCD 0X00000001    ; SQUARE OF 1=1
                    DCD 0X00000004    ; SQUARE OF 2=4
                    DCD 0X00000009    ; SQUARE OF 3=9
                    DCD 0X00000010    ; SQUARE OF 4=16
                    DCD 0X00000019    ; SQUARE OF 5=25
                    DCD 0X00000024    ; SQUARE OF 6=36
                    DCD 0X00000031    ; SQUARE OF 7=49
                    DCD 0X00000040    ; SQUARE OF 8=64
                    DCD 0X00000051    ; SQUARE OF 9=81
                    DCD 0X00000064    ; SQUARE OF 10=100
                    END                ; Mark end of file

```

Result:

6. Write a program to find the largest/smallest number in an array of 32bit numbers (N=7)

```

AREA LARGEST, CODE, READONLY
    ENTRY                ; Mark first instruction to execute
    EXPORT __main
__main                    ; Mark first instruction to execute
    MOV R5, #6           ; INITIALISE COUNTER TO 6(i.e. N=7)
    LDR R1, =VALUE1      ; LOADS THE ADDRESS OF FIRST VALUE
    LDR R2, [R1], #4     ; WORD ALIGN TO ARRAY ELEMENT

LOOP
    LDR R4, [R1], #4     ; WORD ALIGN TO ARRAY ELEMENT
    CMP R2, R4           ; COMPARE NUMBERS
    BHI LOOP1            ; IF THE FIRST NUMBER IS > THEN GOTO LOOP1
    MOV R2, R4           ; IF THE FIRST NUMBER IS < THEN MOVE CONTENT R4 TO R2

LOOP1
    SUBS R5, R5, #1      ; DECREMENT COUNTER
    CMP R5, #0           ; COMPARE COUNTER TO 0
    BNE LOOP            ; LOOP BACK TILL ARRAY ENDS
    LDR R4, =RESULT      ; LOADS THE ADDRESS OF RESULT
    STR R2, [R4]         ; STORES THE RESULT IN R1
    NOP
    NOP
    NOP                  ; ARRAY OF 32-BIT NUMBERS (N=7)

VALUE1
    DCD 0X44444444
    DCD 0X22222222
    DCD 0X11111111
    DCD 0X33333333
    DCD 0XAAAAAAAA
    DCD 0X88888888
    DCD 0X99999999

    AREA DATA2, DATA, READWRITE    ; TO STORE RESULT IN GIVEN ADDRESS
RESULT
    DCD 0X0
    END                                ; Mark end of file
    
```

7. Write a program to arrange a series of 32-bit numbers in ascending/descending order.

```

AREA ASCENDING, CODE, READONLY
ENTRY                               ; Mark first instruction to execute
EXPORT __main

__main                               ; Mark first instruction to execute
    MOV R8,#4                        ; INITIALISE COUNTER TO 4(i.e. N=4)
    LDR R2,=CVALUE                   ; ADDRESS OF CODE REGION
    LDR R3,=DVALUE                   ; ADDRESS OF DATA REGION
LOOP0    LDR R1,[R2],#4               ; LOADING VALUES FROM CODE REGION
        STR R1,[R3],#4               ; STORING VALUES TO DATA REGION
        SUBS R8,R8,#1                ; DECREMENT COUNTER
        CMP R8,#0                    ; COMPARE COUNTER TO 0
        BNE LOOP0                    ; LOOP BACK TILL ARRAY ENDS
START1    MOV R5,#3                    ; INITIALISE COUNTER TO 3(i.e. N=4)
        MOV R7,#0                    ; FLAG TO DENOTE EXCHANGE HAS OCCURED
        LDR R1,=DVALUE               ; LOADS THE ADDRESS OF FIRST VALUE
LOOP1    LDR R2,[R1],#4               ; WORD ALIGN TO ARRAY ELEMENT
        LDR R3,[R1]                  ; LOAD SECOND NUMBER
        CMP R2,R3                    ; COMPARE NUMBERS
        BLT LOOP2                    ; IF THE FIRST NUMBER IS < THEN GOTO LOOP2
        STR R2,[R1],#-4               ; INTERCHANGE NUMBER R2 & R3
        STR R3,[R1]                  ; INTERCHANGE NUMBER R2 & R3
        MOV R7,#1                    ; FLAG DENOTING EXCHANGE HAS TAKEN PLACE
LOOP2    ADD R1,#4                    ; RESTORE THE PTR
        SUBS R5,R5,#1                ; DECREMENT COUNTER
        CMP R5,#0                    ; COMPARE COUNTER TO 0
        BNE LOOP1                    ; LOOP BACK TILL ARRAY ENDS
        CMP R7,#0                    ; COMPARING FLAG
        BNE START1                   ; IF FLAG IS NOT ZERO THEN GO TO START1
        NOP
        NOP
        NOP                           ; ARRAY OF 32 BIT NUMBERS(N=4) IN CODE REGION
CVALUE
    DCD 0X44444444 ;
    DCD 0X11111111 ;
    DCD 0X33333333 ;
    DCD 0X22222222 ;
        AREA DATA1, DATA, READWRITE ; ARRAY OF 32 BIT NUMBERS IN DATA REGION
DVALUE
    DCD 0X00000000 ;
    END                               ; Mark end of file

```

8. Write a program to count the number of ones and zeros in two consecutive memory locations.

```

                AREA SEARCH1, CODE, READONLY
ENTRY           ; Mark first instruction to execute
EXPORT __main
__main          ; Mark first instruction to execute
                MOV r5,#2
                MOV r0,#0
                MOV r1,#0
                LDR r2, VALUE1
SECOND
NEXT            MOV r3, #32
                MOVS r2,r2,ror #1
                BLO ZERO
                ADD r1,#1
                B LAST
ZERO
LAST           ADD r0,#1
                SUBS r3,#1
                BNE NEXT          ; Branch to the loop if not equal to Zero
                LDR r2, VALUE2
                SUBS r5,#1
                BNE SECOND
                STOP B STOP
                NOP
VALUE1         DCD 0X1234567A    ; GIVEN FIRST NUMBER IN MEMORY
VALUE2         DCD 0X1234567A    ; GIVEN SECOND NUMBER IN MEMORY
                END              ; Mark end of file
    
```


PART_B

INTERFACING PROGRAMS

Hardware Programs:

Register Configuration

LPC1768 has its GPIOs divided into five ports PORT0 - PORT4, although many of them are not physically 32bit wide. Refer the data sheet for more info. The Below registers will be used for Configuring and using the GPIOs registers for sending and receiving the Digital signals.

PINSEL: GPIO Pins Select Register

Almost all the LPC1768 pins are multiplexed to support more than 1 function. Every GPIO pin has a minimum of one function and max of four functions. The required function can be selected by configuring the PINSEL register. As there can be up to 4 functions associated with a GPIO pin, two bits for each pin are available to select the function. This implies that we need two PINSEL registers to configure a PORT pins. By this the first 16(P0.0-P0.15) pin functions of PORT0 can be selected by 32 bits of PINSEL0 register. The remaining 16 bits (P0.16-P0.31) are configured using 32bits of PINSEL1 register. As mentioned earlier every pin has max of four functions. Below table shows how to select the function for a particular pin using two bits of the PINSEL register.

Value	Function	Enumeration
00	Primary (default) function, typically GPIO port	PINSEL_FUNC_0
01	First alternate function	PINSEL_FUNC_1
10	Second alternate function	PINSEL_FUNC_2
11	Third alternate function	PINSEL_FUNC_3

FIODIR: Fast GPIO Direction Control Register.

This register individually controls the direction of each port pin.

Values	Direction
0	Input
1	Output

FIOSET: Fast Port Output Set Register.

This register controls the state of output pins. Writing 1s produces highs at the corresponding port pins. Writing 0s has no effect. Reading this register returns the current contents of the port output register not the physical port value.

Values	FIOSET
0	No Effect
1	Sets High on Pin

FIOCLR: Fast Port Output Clear Register.

This register controls the state of output pins. Writing 1s produces lows at the corresponding port pins. Writing 0s has no effect.

Values	FIOCLR
0	No Effect
1	Sets Low on Pin

FIOPIN: Fast Port Pin Value Register.

This register is used for both reading and writing data from/to the PORT.

Output: Writing to this register places corresponding values in all bits of the particular PORT pins.

Input: The current state of digital port pins can be read from this register, regardless of pin direction or alternate function selection (as long as pins are not configured as an input to ADC).

Note: It is recommended to configure the PORT direction and pin function before using it.


```

        //For Anticlockwise
        for(k=0;k<15000;k++);
    variation
    }
}
//for step speed

```

10. Interface a DAC and generate Triangular and Square waveforms.

Square wave:

```

#include "LPC17xx.h"
void delay(unsigned int ms) // delay subroutine
{
    unsigned int i,j;
    for(i=0;i<ms;i++)
        for(j=0;j<2000;j++);
}
int main()
{
    SystemInit();
    LPC_PINCON->PINSEL1 |= 0x02<<20; //p0.26 pinsel bits 20 and 21 while(1)
    {
        LPC_DAC->DACR=0xffff; // send maximum values to dac
        delay(20); // delay
        LPC_DAC->DACR=0x0000; // send min values to dac
        delay(20);
    }
}

```

Result:

Triangular wave:

```
#include "LPC17xx.h"
uint32_t val;
int main()
{
    SystemInit();
    LPC_PINCON->PINSEL1 |= 0x02<<20; //p0.26 pinsel bits 20 and 21 while(1)
    {
        while(1)
        {
            val++; // increment values by 6
            LPC_DAC->DACR=(val<<6); //send values to dac
            if(val>=0x3ff) // if value exceeds 1024
            {
                break;
            }
        }
        while(1)
        {
            val--; // decrement value by 6
            LPC_DAC->DACR=(val<<6); // send value to dac
            if(val<=0x000) // if value come down by 0
            {
                break;
            }
        }
    }
}
```

Result:

11. Display the Hex digits 0 to F on a 7-segment LED interface, with a suitable delay in between.

```

/* Description :\\repeatedly\\DISPLAY ARE CONNECTED IN COMMON CATHODE
MODE\\repeatedly\\
Port0 Connected to data lines of all 7 segment displays
a
----
f| g |b
|---|
e| |c
---- . dot
d
a = P0.04
b = P0.05
c = P0.06
d = P0.07
e = P0.08
f = P0.09
g = P0.10
dot = P0.11
*/

#include <LPC17xx.h>
unsigned int delay, count=0, Switchcount=0,j;

unsigned int Disp[16]={0x000003f0, 0x00000060, 0x000005b0, 0x000004f0,
0x00000660,0x000006d0,0x000007d0, 0x00000070, 0x000007f0, 0x000006f0,
0x00000770,0x000007c0,0x00000390, 0x000005e0, 0x00000790, 0x00000710 };

#define ALLDISP 0x00180000 //Select all display
#define DATAPORT 0x00000ff0 //P0.4 to P0.11 : Data lines connected to
drive Seven Segments

int main (void)
{
    LPC_PINCON->PINSEL0 = 0x00000000;
    LPC_PINCON->PINSEL1 = 0x00000000;
    LPC_GPIO0->FIODIR = 0x00180ff0;

    while(1)
    {
        LPC_GPIO0->FIOSET |= ALLDISP;
        LPC_GPIO0->FIOCLR = 0x00000ff0;
        // clear the data lines to 7-segment displays
        LPC_GPIO0->FIOSET = Disp[Switchcount]; // get the 7-segment display
                                                value from the array

        for(j=0;j<3;j++)
            for(delay=0;delay<30000;delay++); // delay

        Switchcount++;
        if(Switchcount == 0x10)
            // 0 to F has been displayed ? go back to 0
    }
}

```

```

        Switchcount = 0;
        LPC_GPIO0->FIOCLR = 0x00180ff0;

    }

}

```

12. Interface a simple Switch and display its status through Relay, Buzzer and LED.

```

#include <LPC17xx.H>
int main(void)
{
    LPC_PINCON->PINSEL1 = 0x00000000;    //P0.24,P0.25 GPIO
    LPC_GPIO0->FIODIR = 0x03000000;    //P0.24 configured output for
    buzzer,P0.25 configured output for Relay/Led
    while(1)
    {
        if(!(LPC_GPIO2->FIOPIN & 0x00000800))    //Is GP_SW(SW4) is pressed??
        {
            LPC_GPIO0->FIOSET = 0x03000000; //relay on
        }
        else
        {
            LPC_GPIO0->FIOCLR = 0x03000000; //relay off
        }
    }
}

//end int main(void)

```